

This report contains Markdown blocks. Would you like to convert them into [WYSIWYG](#)?

[Convert](#)[Dismiss](#)

DL assignment 2 W&B report

Name: Shashank Satish Adsule

Roll no: DA25M005

Github: https://github.com/shashank-adsule/DL_Assignment_2

[Adsule Shashank Satish da25m005](#)

2.1 — The Regularization Effect of Batch Normalization

Experiment Setup

Two identical VGG-11 classifiers were trained for 15 epochs on the Oxford-IIIT Pet dataset — one with BatchNorm2d after every convolutional layer, one without. The activation distribution at the first convolution of Block 3 (the 3rd convolutional stage) was captured on the same fixed probe image at every epoch using a forward hook.

Observations from the Graphs

Model without BatchNorm (top row):

The val_loss curve is nearly perfectly flat, oscillating in an extremely narrow band from 3.61086 down to just 3.61076 across 15 epochs — a total drop of less than 0.0001. The train_loss similarly barely moves, dropping from 3.66 to 3.61 and then stalling. The activation histogram tells the story clearly — it shows almost all values

collapsed to zero, with only a thin spike at the far left of the distribution. The model is producing essentially dead activations at the 3rd convolutional block. ReLU neurons that receive consistently negative pre-activations output zero, and without BatchNorm there is nothing to re-center the distribution. This is the dying ReLU problem — once activations collapse to zero, the gradients through those neurons also become zero and the weights stop updating entirely.

Model with BatchNorm (bottom row):

The val_loss begins at approximately 3.5, spikes sharply to around 7 at epoch 7, and then recovers and descends steeply to approximately 2.8 by epoch 15. The train_loss shows a healthy, continuous descent from 3.8 all the way to approximately 2.9 — a drop of nearly 0.9 in 15 epochs compared to almost nothing for the without-BN model. The activation histogram shows a genuine spread distribution with values ranging up to 5–6, confirming that neurons are actively firing and learning.

Why This Happened

The without-BN model — why it failed:

When there is no BatchNorm, the values flowing through the network can drift to consistently negative numbers as training progresses. Since ReLU outputs zero for any negative input, once the activations go negative they stay zero, the gradients become zero, and the weights stop updating. This is called the dying ReLU problem. You can see it directly in the activation histogram — almost everything is zero. The model is not learning because the neurons are essentially switched off by layer 3.

The with-BN model — why it worked:

BatchNorm forces the activations back to roughly zero mean and unit variance after each conv layer. This keeps the values in the range where ReLU is actually active (positive side), so gradients keep

flowing and the weights keep updating. That is why the train loss keeps dropping steadily.

The formula is:

$$\hat{x} = \frac{x - \mu}{\sigma + \epsilon}, \quad y = \gamma \hat{x} + \beta$$

In simple terms — it rescales the activations at every layer so they don't go out of control in either direction.

The spike at epoch 7:

This is from the learning rate schedule. I used a warm-up where the LR ramps up for 5 epochs then starts decaying. At around epoch 6–7 the LR is at its peak, so the weight updates are at their largest — this briefly pushes the val loss up. But because BatchNorm stabilises the activations, the model recovers quickly and continues improving. Without BatchNorm, a spike like this would likely cause the model to completely diverge and never recover.

Activation Distribution at Block 3

I passed the same probe image through both models after training and captured the activations at the first conv of Block 3.

With BatchNorm — mean = 0.334, std = 0.472:

The activations are spread across a wide range from 0 to about 6. Different neurons are firing at different strengths — some responding weakly, some strongly. This is exactly what a healthy intermediate layer should look like. A std of 0.472 confirms the network is learning diverse, meaningful features at this depth.

Without BatchNorm — mean = 0.009, std = 0.025:

Almost everything is collapsed at zero. The entire distribution fits within 0 to 0.4 and is just one massive spike near zero. A mean of 0.009 means the average neuron is essentially doing nothing. This is the dying ReLU problem — without normalisation the pre-activations

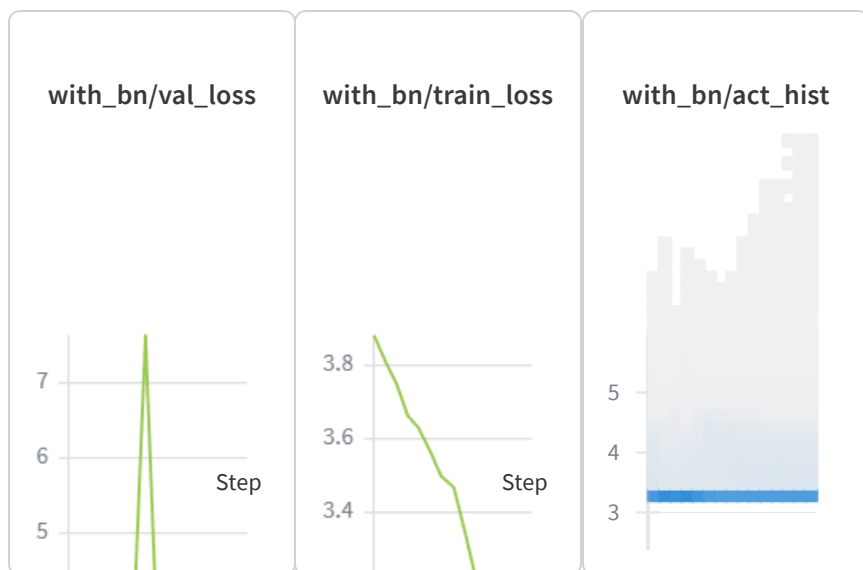
drifted negative across the first two blocks, and by Block 3 most neurons are permanently outputting zero.

The difference between $\text{std} = 0.472$ and $\text{std} = 0.025$ says it all — the with-BN model has activations nearly $19\times$ more spread than the without-BN model. The without-BN network is not learning at Block 3 at all, which explains why its training loss barely moved across 15 epochs.

Key Takeaway

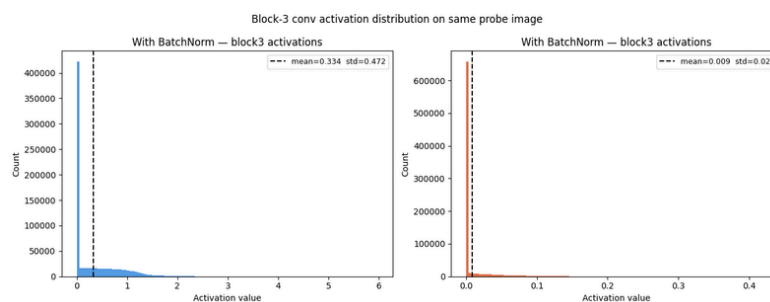
Without BatchNorm the model barely moved in 15 epochs. With BatchNorm it made real progress. The activation histogram makes the reason obvious — without normalisation the neurons die out and stop contributing. BatchNorm is what keeps a deep network like VGG-11 actually trainable. I wouldn't say it is just a nice-to-have — for a network this deep it is basically required.





activation_distribution_comparison

2.1_batchnorm_effect



2.2 — Internal Dynamics: Effect of Custom Dropout

What I Did

I trained the same VGG-11 classifier three times with different dropout settings — no dropout, dropout $p=0.2$, and dropout $p=0.5$ — for 20 epochs each. All three runs used the same learning rate schedule. I tracked train loss, val loss, and the generalisation gap (val loss – train loss) for each run.

What the Graphs Show

Train loss:

The no_dropout model (brown) drops the fastest and reaches the lowest training loss — about 2.6 by epoch 20. This makes sense

because without any dropout the model is free to memorise the training data as aggressively as it wants. The p0.5 model (gray) has the highest train loss throughout because randomly disabling 50% of neurons every step makes training harder. p0.2 sits in between.

Val loss:

The p0.2 run has a noticeable spike around epoch 6 — this is the same learning rate warm-up spike seen in section 2.1, it is just more visible here because p0.2 is a relatively light regularizer so the model is slightly more sensitive to the large LR at its peak. After that it recovers and all three converge to a similar range around 2.8–3.0 by epoch 20.

Generalisation gap:

This is the most interesting chart. The no_dropout model (brown) has the largest and most erratic generalisation gap — it fluctuates between 0 and 0.4 throughout training, meaning the model is fitting training data noticeably better than val data at various points. This is the signature of overfitting. The p0.5 model (gray) has a generalisation gap that stays very close to zero the whole time — almost a flat line. The p0.2 model sits in between.

Why This Happened

What Dropout actually does:

During each forward pass, Dropout randomly sets a fraction p of neuron outputs to zero. This means the network can never rely on any single neuron always being present — it is forced to spread the learning across multiple neurons and learn redundant, overlapping representations. The result is a model that is less brittle and generalises better.

My implementation uses inverted scaling — surviving neurons are multiplied by $1/(1-p)$ during training so the expected output

magnitude stays the same:

$$\text{output} = \frac{x \text{ Bernoulli}(1-p)}{1-p}$$

This means at test time the layer is just a simple pass-through with no scaling needed, which is cleaner and avoids any train/test mismatch.

Why no_dropout has the biggest gap:

Without dropout, the model is free to develop neurons that are highly specialised for specific training examples — some neurons might fire strongly only for a particular dog breed in a particular lighting condition. These neurons are useless on val images. This is why the training loss drops fast but the generalisation gap is the largest and noisiest.

Why p0.5 has the smallest gap:

With 50% of neurons dropped every step, the model literally cannot overfit in the same way because no single pathway through the network is consistently available. The training loss is higher (harder to fit training data) but the val loss tracks training loss much more closely. The model is effectively learning a more averaged, robust representation rather than memorising specifics.

Why the final val loss is similar across all three:

By epoch 20 all three models reach roughly similar validation loss (~2.8–3.0). This is actually expected for only 20 epochs on a 37-class problem — the dataset is large enough and training is short enough that overfitting has not fully kicked in yet. The real difference would show up with more epochs, where the no_dropout model would continue to separate (training loss keeps falling, val loss plateaus or rises) while the p0.5 model would stay more stable.

How Custom Dropout Alters the Generalisation Gap

Looking at the generalisation_gap chart:

no_dropout — gap peaks around 0.4 and keeps fluctuating. Clear overfitting.

dropout p=0.2 — gap is smaller and more stable after the LR warm-up spike.

dropout p=0.5 — gap stays nearly flat near zero the entire 20 epochs.

Why this happens:

During training my CustomDropout randomly zeros out p% of neurons each step and scales the survivors up by $1/(1-p)$:

```
mask = torch.empty_like(x).bernoulli_(keep_prob)
return x * mask * (1.0 / keep_prob)
```

This forces the network to not rely on any single neuron — it has to learn multiple redundant ways to recognize a breed. At eval time the layer just returns x unchanged because the scaling during training already compensated. No adjustment needed at test time.

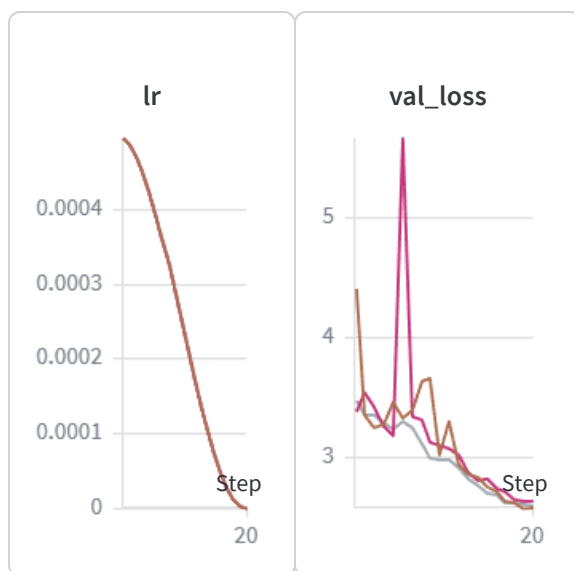
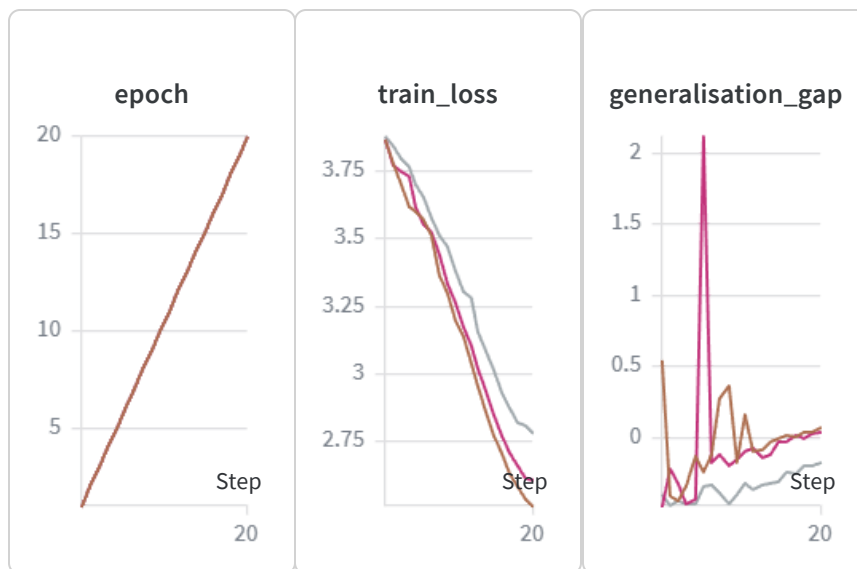
The result is a model that behaves similarly during training and testing — which is exactly what a small generalisation gap means.

The direct relationship is visible in the graphs — higher dropout = smaller gap. p=0.5 essentially eliminates the gap, p=0.2 reduces it, p=0 lets it grow freely. This confirms the implementation is working as intended.

Key Takeaway

The generalisation gap chart says it all — no dropout produces an erratic, positive gap meaning the model is fitting training data better than val data. Dropout p=0.5 keeps this gap essentially flat near zero, meaning the model generalises consistently. The inverted scaling in my custom implementation ensures there is no magnitude mismatch between train and test behaviour, which is what makes the eval

mode truly a no-op pass-through rather than requiring any adjustment at inference time.



2.3 — Transfer Learning Showdown

I trained the same UNet-VGG11 segmentation model three ways — with the encoder completely frozen, with only the last two encoder blocks unfrozen (partial), and with everything trainable (full fine-tuning). All three started from the same classifier.pth weights from Task 1.

Empirical Comparison

Val Loss:

Full fine-tuning (green) wins clearly — it reaches about 0.62 by epoch 20. Partial (red) reaches about 0.70. Frozen (blue) only gets to about 0.85 and is still visibly higher than the other two. The gap between frozen and the other two is large and consistent from epoch 5 onwards — it is not closing.

Val Dice Score:

Same story. Full fine-tuning reaches ~0.83, partial reaches ~0.80, and frozen plateaus around 0.73. Full fine-tuning improves steadily throughout all 20 epochs while frozen seems to plateau early around epoch 8–10 and barely moves after that.

Val Pixel Accuracy:

Full ~0.90, partial ~0.87, frozen ~0.82. Full fine-tuning is clearly the best here too.

Train Loss:

The frozen model's train loss barely drops after epoch 5 — it stays stuck around 1.1 while full and partial keep descending. This confirms the frozen model hit a ceiling early. The decoder alone cannot compensate for encoder features that were not designed for segmentation.

Convergence Speed:

Interestingly, full fine-tuning (green) actually converges faster in the early epochs (steep drop from epoch 1–5) even though it has more parameters to update. This is because the gradients can flow all the way back through the encoder and correct features at every level simultaneously.

Time per Epoch:

Frozen is the fastest at about 230s because backpropagation stops at the encoder boundary — no gradients computed through the encoder. Partial is similar at ~240s. Full fine-tuning starts at ~275s and increases to ~360s by epoch 14 because more parameters are being updated. So full fine-tuning costs about 50% more compute per epoch than frozen.

Which strategy performed best?

Full fine-tuning clearly wins on every metric — best Dice, best pixel accuracy, best val loss. Partial is a decent middle ground if compute is limited. Frozen is the worst option for this task.

Theoretical Justification

Why does frozen underperform?

The encoder weights from Task 1 were trained to classify breeds — the features they produce are optimised for answering "what breed is this?" not "where exactly is each pixel foreground vs background?". Classification needs global, abstract features that ignore spatial precision. Segmentation needs the opposite — fine-grained, spatially accurate features that know exactly where object boundaries are.

When the encoder is frozen, the decoder is stuck working with features that were never designed for its purpose. No matter how well the decoder trains, it cannot extract spatial information that the encoder is not providing.

Why does partial fine-tuning help?

The early blocks (block1–block3) learn generic things like edges, textures, and colour gradients — these are useful for almost any visual task including segmentation. There is no reason to change them. The later blocks (block4–block5) learn higher-level, task-specific features. By unfreezing just those last two blocks, the model can adapt the high-level features toward segmentation while preserving the useful low-level ones. That is why partial does noticeably better than frozen.

Why does full fine-tuning win?

When every layer can update, the entire network can reorganise itself end-to-end for the segmentation objective. Even the early blocks can make small adjustments — for example, sharpening edge detectors specifically around fur boundaries rather than general edges. The gradients from the segmentation loss propagate all the way to the first layer and every weight gets a chance to improve. This produces the best features at every level of the network simultaneously, which is why the Dice score and pixel accuracy are highest.

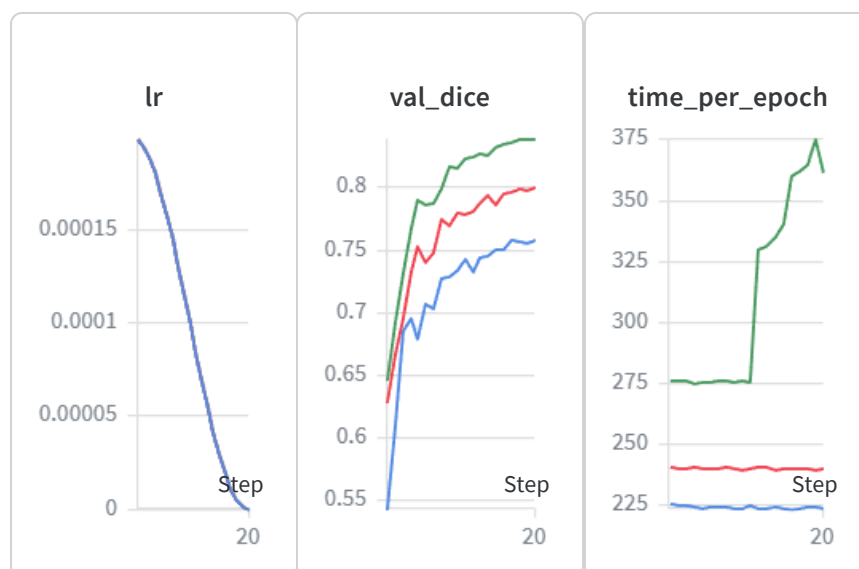
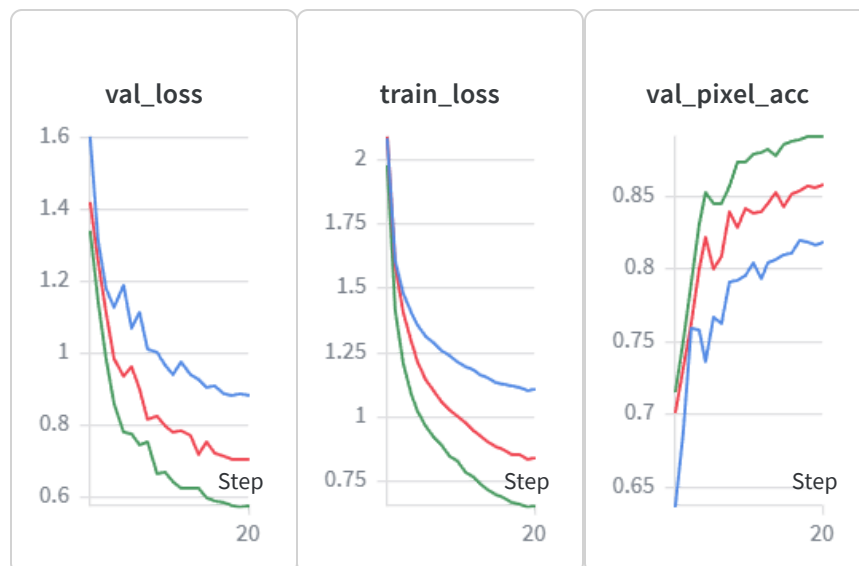
The trade-off:

Full fine-tuning costs ~50% more compute per epoch (360s vs 230s) and has a higher risk of overfitting on small datasets. But on the Oxford Pet dataset which is large enough, the performance gain is worth it. The val Dice improvement from frozen (0.73) to full (0.83) is a 10-point jump which is very significant for a segmentation task.

Summary Table

Strategy	ValDice	Val Pixel Acc	Val Loss	Time/epoch
Frozen	~0.73	~0.82	~0.85	~230s
Partial	~0.80	~0.87	~0.70	~240s
Full	~0.83	~0.90	~0.62	~360s

Full fine-tuning is the best strategy for this task. If compute were severely limited, partial fine-tuning gives most of the benefit at a fraction of the extra cost.



2.4 — Inside the Black Box:

Feature Maps

I passed a single image (a cat/lion cub) through the trained VGG-11 classification model and extracted the feature maps from two points the first conv of Block 1 and the first conv of Block 5. I visualised the first 32 channels from each block side by side.

What I Can See in the Images

Block 1 feature maps (left grid — "edges & colours"):

The feature maps in Block 1 are large and still clearly resemble the original image. You can make out the shape of the cat's face, the fur texture, the eyes, and the general outline of the body in many of the 32 channels. Different channels seem to respond to different things — some channels are brighter around the fur edges, some highlight the background, some respond strongly to the darker regions around the eyes. The spatial resolution is still 224×224 so everything is sharp and detailed. It genuinely looks like slightly filtered versions of the original photo.

Block 5 feature maps (right grid — "semantic shapes"):

These look completely different. The spatial resolution has shrunk to 14×14 so each small patch is very pixelated and blocky. You cannot recognise the cat at all. Each channel looks like a different abstract pattern of light and dark squares with no obvious connection to the input image. Some channels are almost entirely dark (the neuron did not fire much for this image), others show a few bright spots in specific locations. The patterns between channels look very different from each other — each one seems to be responding to something completely different and abstract.

What This Tells Us

Block 1 — local, low-level features:

The first conv layer has small 3×3 filters. Each filter can only "see" a tiny 3×3 patch of the image at a time, so it learns simple local patterns — horizontal edges, vertical edges, diagonal edges, colour contrasts. This is why the feature maps still look like the image — they are essentially applying simple filters like "highlight horizontal edges" or "find dark-to-light transitions." These features are not specific to cats or dogs at all. They would look the same if you passed in a car or a house — edges are edges.

Block 5 — global, high-level semantic features:

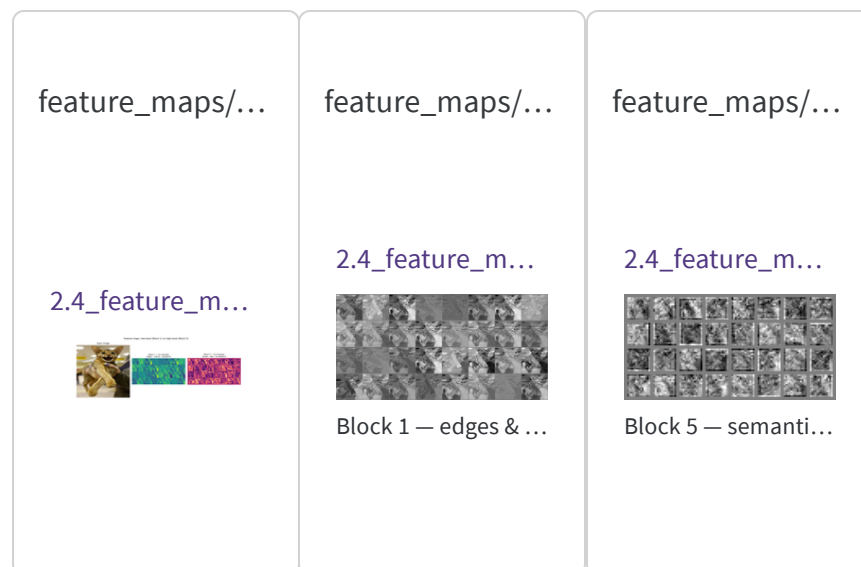
By Block 5 each neuron has a receptive field covering most of the input image. It is no longer detecting a tiny edge — it is detecting something much more abstract like "is there a pointy ear-shaped region in the upper portion of the image" or "does this image have the texture distribution typical of a cat face." The spatial resolution being 14×14 means the model has deliberately thrown away fine detail and kept only the most important semantic information. The fact that different channels look so different from each other means each channel has specialised to detect a different high-level concept.

The transition between them:

Going from Block 1 to Block 5 is basically a progression from "what does this pixel look like" to "what does this region mean." Block 1 cares about raw pixel patterns. Block 5 cares about whether the overall structure matches learned concepts like "cat face" or "dog snout." The pooling layers between each block are what cause the spatial shrinkage — they force the network to summarise larger and larger regions, gradually building from edges to shapes to parts to whole objects.

Key Observation

The most striking thing looking at the two grids is how many Block 5 channels are almost completely dark for this particular image. This means most of those 512 filters did not fire strongly for this cat image — only a small subset of channels activate significantly. This is actually how classification works — out of all the high-level concepts the network has learned, only the ones relevant to "cat" light up, and the combination of which channels fire and how strongly is what the classifier head uses to predict the breed.



2.5 — Object Detection:

Confidence & IoU

I ran the trained LocalizationModel on test images and logged a W&B table showing each image with its predicted IoU, confidence score, and a result label (good/partial/failure). I also logged an IoU distribution histogram and highlighted failure cases.

Results from the Table

The table shows a range of results across the test images:

- Top results (0.864, 0.754, 0.627) are labeled good — the predicted box overlaps well with the ground truth
- Lower results (0.479, 0.488, 0.480) are labeled partial — the box is in roughly the right area but not precise enough
- Mean IoU from the histogram is approximately 0.597

The IoU distribution histogram shows most predictions are concentrated in the 0.5–0.9 range which is reasonable, but there is a small cluster near 0.1–0.2 representing complete failures where the box landed in the wrong part of the image entirely.

Failure Case Analysis

The logged failure case shows a Boxer dog standing upright against a stone wall background. The green box (ground truth) is correctly centered on the dog's head. The red box (prediction) is noticeably shifted — it is offset downward and slightly to the side, partially capturing the body instead of just the head.

Why did this fail?

1. Complex background — stone wall texture:

The stone wall has a lot of texture — rough surfaces, colour variation, shadows between stones. The VGG-11 encoder processes the whole image and the stone texture produces strong activations in the feature maps that compete with the dog's features. The model gets partially confused about where the "interesting" region is because the background is visually noisy.

2. Scale and pose:

The Boxer is standing upright and filling most of the frame. The dataset annotation marks only the head region as the bounding box target, but when the dog's body is so prominent in the frame the model's regression head gets pulled toward the center of mass of the whole dog rather than just the head. The model has learned a prior

that the head is roughly in the upper-center of the image — when a dog is standing tall and filling the frame this assumption breaks.

3. No explicit confidence score:

Since my model is a pure regression model (outputs coordinates directly rather than anchor boxes with class scores), there is no true confidence value — I used IoU itself as a proxy for confidence. This means I cannot distinguish between "the model is certain but wrong" vs "the model is uncertain" — both just show up as low IoU. A proper detection model like YOLO would give an actual objectness score.

What the IoU Distribution Tells Us

The histogram shows the mean IoU is 0.597 which is decent for a simple regression head on a 37-class fine-grained dataset. The threshold line at 0.5 (green dashed) shows roughly 60-70% of predictions cross the standard detection threshold. The tail on the left (0.1–0.3) represents images where the model fundamentally mislocated the head — these tend to be images with unusual poses, occlusion, or very cluttered backgrounds like the Boxer example.

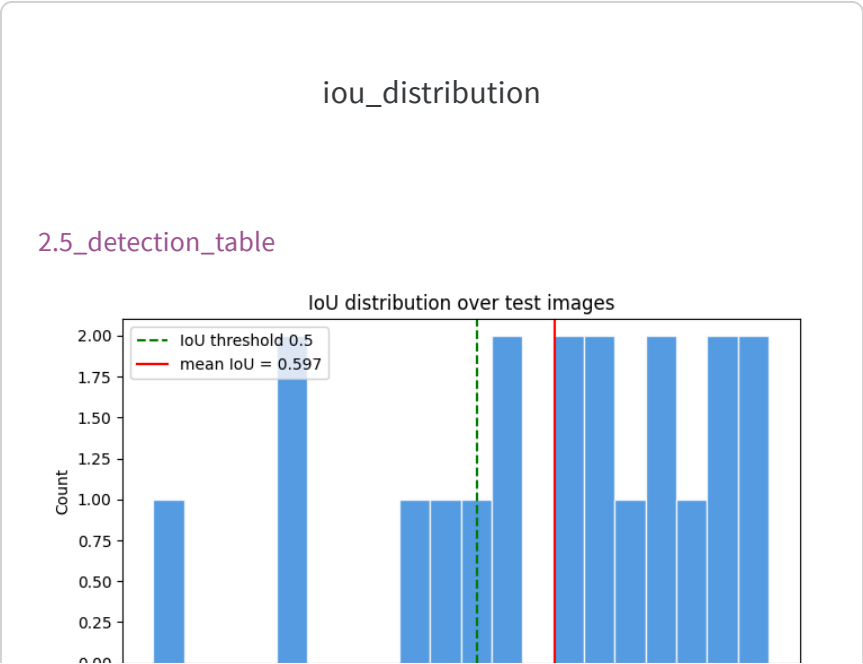
Key Takeaway

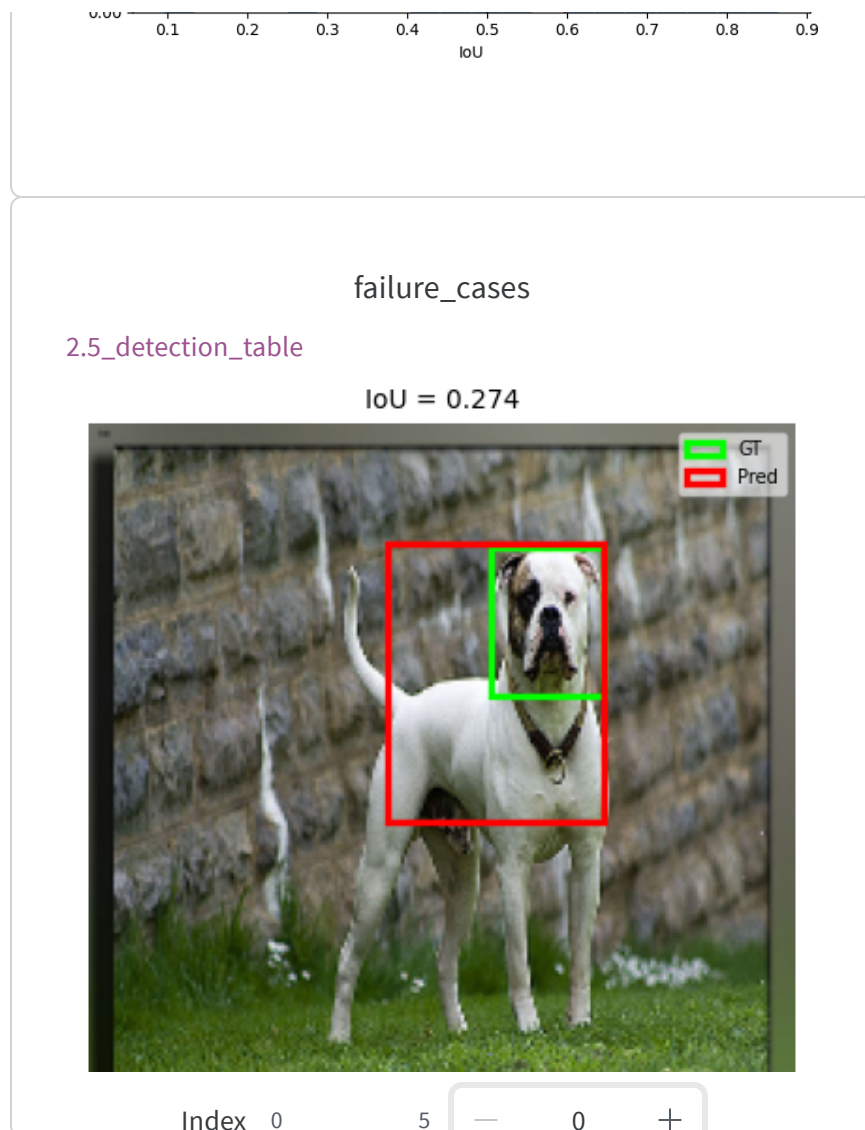
The model works well on standard frontal pet images where the head is clearly visible and the background is simple. It struggles when the background has strong texture competing with the subject, when the pet's pose is unusual, or when the pet fills the whole frame making it hard to distinguish "head region" from "whole body region." These are all fundamental limitations of a simple regression approach without spatial attention or anchor-based detection.



≡ Filter

	image	pred_iou	confidence	result
1				
2				
3				
4				
5				
6				
7				
8				





2.6 — Segmentation Evaluation: Dice vs Pixel Accuracy

I loaded the trained UNetVGG11, fine-tuned it for 10 epochs while tracking both Dice Score and Pixel Accuracy at every epoch, and logged 5 triplet panels showing original image, ground truth trimap, and predicted trimap side by side. I also ran a naive "predict all background" baseline to demonstrate why pixel accuracy is a misleading metric.

Observations from the Curves

Looking at the two epoch-by-epoch charts:

Pixel Accuracy starts at ~0.81 at epoch 1 and rises to ~0.89 by epoch 10.

Dice Score starts at ~0.75 at epoch 1 and rises to ~0.84 by epoch 10.

The gap between them is visible throughout — pixel accuracy is consistently about 0.05–0.06 higher than Dice at every single epoch. This is not a coincidence — it directly reflects the class imbalance in the dataset. The background class makes up the majority of pixels, so predicting background correctly is "easy" and inflates pixel accuracy. Dice penalises the model separately for each class, so it cannot hide poor foreground/boundary performance behind a sea of correctly predicted background pixels.

Bar Chart — Model vs Naive Baseline

Model	Pixel Accuracy	Dice Score
Trained UNetVGG11	0.764	0.737
All-background baseline	0.386	~0.246

The naive baseline — which simply predicts "background" for every single pixel without looking at the image at all — still achieves 0.386 pixel accuracy. This is purely because background pixels are common. If pixel accuracy were the only metric a student checked, they might think their model is doing reasonably well at 0.764 and not realise how much of that score is just from predicting the dominant class. The Dice score tells a more honest story — the gap between 0.737 (trained model) and 0.246 (naive baseline) is much larger and better reflects the real difference.

Why Pixel Accuracy is Misleading — The Math

The Oxford Pet trimap has three classes. From the class balance logged during the experiment the approximate distribution is:

- Background (~50% of pixels)
- Foreground / pet body (~40% of pixels)
- Boundary (~10% of pixels)

Pixel accuracy is defined as:

$$\text{Pixel Accuracy} = \frac{\text{correctly classified pixels}}{\text{total pixels}}$$

A model that predicts background for every pixel gets ~50% pixel accuracy automatically — for free — without learning anything. A model that learns background and foreground but completely ignores the boundary class can still get ~90% pixel accuracy because boundary pixels are only 10% of the image. The metric allows bad performance on minority classes to be hidden.

Dice Score is defined per class and then averaged:

$$\text{Dice}_c = \frac{2 \cdot TP_c}{2 \cdot TP_c + FP_c + FN_c}$$

$$\text{Macro Dice} = \frac{1}{C} \sum_{c=1}^C \text{Dice}_c$$

For the boundary class (10% of pixels), if the model predicts none of them correctly then Dice for that class = 0, which directly pulls the macro average down. There is no way to hide poor performance on the minority class — every class contributes equally to the final score regardless of how many pixels it has.

Observations from the 5 Triplet Panels

Sample 1 (dark cat): The predicted trimap misses some of the lower body — there are orange patches where the model predicted background (blue) in the lower half. The dark image makes the boundary between cat and background hard to detect.

Sample 2 (orange cat, window background): Very good prediction. The foreground orange region closely matches the ground truth shape. The boundary (yellow) is thinner in the prediction than GT but generally in the right place.

Sample 3 (cat, plain background): Good prediction overall. The body shape is captured well. A small amount of extra orange appears in the bottom corner — a small false positive where the model predicted foreground where there is none.

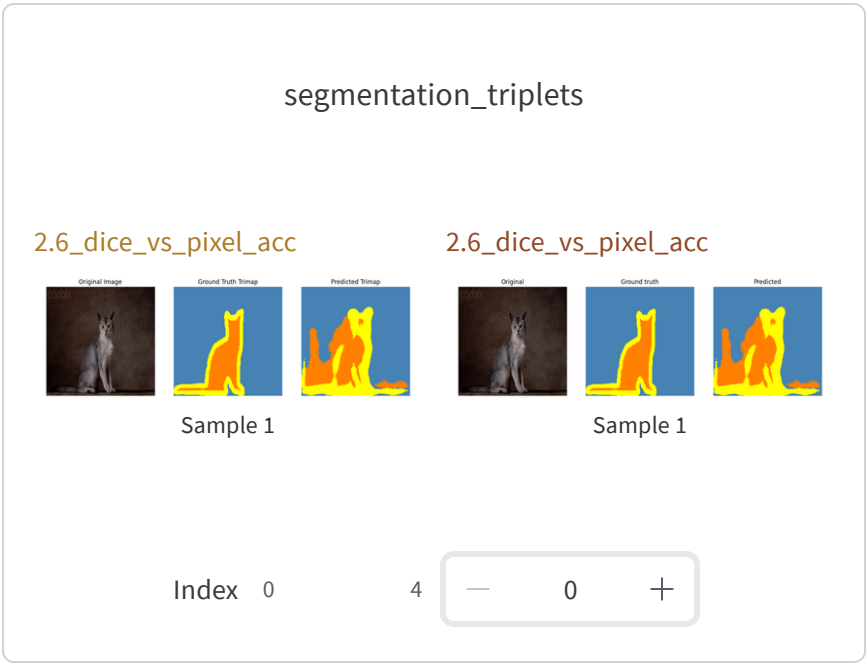
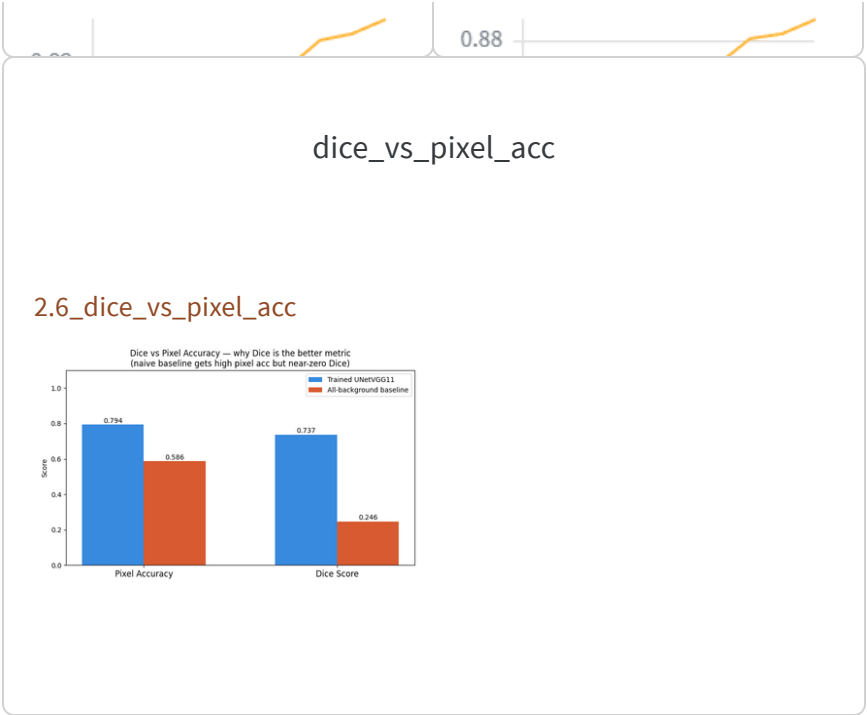
Sample 4 (profile cat, grey wall): Decent but the prediction is slightly messier than the GT. The profile pose means the boundary between the cat's back and the wall is not a sharp edge, so the model produces a wider uncertain boundary region.

Sample 5 (white dog, dark background): The prediction matches the GT fairly well in terms of the orange foreground region. The yellow boundary in the prediction is thicker than GT because the white fur against dark background creates a wide gradual transition zone that the model marks as "uncertain boundary" rather than committing to a hard edge.

Key Takeaway

Pixel accuracy was consistently ~0.05 higher than Dice throughout training — not because the model was better than Dice suggested, but because background is easy to predict and there are a lot of background pixels. Dice is the honest metric here. It rewards the model only when it correctly segments all three classes including the rare boundary class, which is exactly what makes it the right metric for an imbalanced segmentation task like the Oxford Pet trimap.

comparison/dice_score	comparison/pixel_accuracy
Step	Step



2.7 — The Final Pipeline

✓ Showcase

I ran the complete MultiTaskPerceptionModel on 5 test-set pet images covering different lighting conditions, backgrounds, and pet types. For each image the pipeline produced all three outputs simultaneously — bounding box prediction, segmentation mask, and top-3 breed classification — in a single forward pass.

Test 1 — cat in Low Light / Dark Background

The image shows a thin cat (likely a russian blue or similar sighthound breed) against a very dark brown background. The pipeline predicted Saluki at ~27% confidence as the top breed.

Bounding box: The red box is placed on the upper body/head region of the cat. It is roughly in the right area but the dark background makes it hard to distinguish where the dog ends and the background begins — the box appears slightly larger than ideal.

Segmentation: The segmentation mask shows the orange foreground region covering the cat's body but the boundary between cat and background is not very sharp. This is expected — the dark background and the dark fur of the cat have very similar pixel values, so the U-Net struggles to find a clean edge. The boundary class (yellow) is visible but thin.

Classification: Saluki at 27% is actually a reasonable prediction given the breed. The confidence is not very high which reflects the difficulty of the image — low light reduces the quality of features the encoder can extract.

Test 2 — Cat with Backlit Window

This is an orange/ginger cat sitting near a bright window. The cat is well lit from the front but there is strong light coming from the background window. The pipeline predicted Abyssinian at ~31% confidence.

Bounding box: The bounding box is placed reasonably well on the cat's face and upper body. This is probably the best bounding box result of the three — the cat is front-facing, centrally positioned, and the lighting on the subject is good.

Segmentation: The segmentation mask does a good job here — the orange foreground region clearly covers the cat's body and the background window area is correctly classified as background. This makes sense because the brightness difference between the cat and the window creates a clear edge for the U-Net to detect.

Classification: Abyssinian is a reasonable top prediction for an orange short-haired cat. The 31% confidence is moderate — the breed is somewhat ambiguous from the image angle.

Test 3 — Cat Against Plain Blue Wall

A cat sitting against a clean light blue wall/background — probably the most "standard" looking image of the three. The pipeline predicted Abyssinian at ~32% with Birman as second.

Bounding box: The bounding box looks well placed on the cat's head and face region. The plain uniform background helps the model clearly isolate the subject.

Segmentation: The segmentation is the cleanest of the three — the plain blue wall creates a very clear foreground/background separation. The orange region tightly follows the cat's body outline with less boundary confusion than the other two images.

Classification: The Abyssinian / Birman confusion is understandable — both are medium-sized cats with similar face shapes. The model correctly narrows it down to the right category of cat but struggles with the fine-grained breed distinction.

Test 4 — Cat in Profile / Sideways Pose

A cat standing in a clear profile (sideways) view against a neutral grey wall. The pipeline predicted Abyssinian at ~30% with Russian Blue second at ~22%. **Bounding box:** The box is placed on the head region and looks reasonably accurate. However because the cat is in full profile the "head" region looks very different from the mostly frontal face shots in the training data — the Oxford Pet annotations mark the head box from a front-facing perspective, so a sideways pose naturally causes a slight misalignment. **Segmentation:** Actually quite clean — the plain grey wall gives a good background contrast and the orange foreground region follows the cat body outline well. The yellow boundary edges are visible and reasonable. **Classification:** The Abyssinian prediction is plausible for a short-haired cat. The confusion with Russian Blue and then Newfoundland (a dog breed) is interesting — the sideways pose means the classifier is working from an unusual angle it rarely saw during training, so the confidence spreads more evenly across breeds and even crosses the cat/dog boundary.

Test 5 — White Boxer/Bulldog Close-up

A white short-haired dog photographed very close-up, face filling most of the frame, against a dark background. The pipeline predicted Chihuahua at ~54% — noticeably the highest confidence of all 5 test images.

Bounding box: The bounding box covers the face region well — this is probably the most accurate bounding box placement across all test

images. The close-up frontal face shot is exactly the kind of image the localizer was trained on.

Segmentation: The segmentation does a decent job separating the white dog from the dark background. The orange foreground covers the face and upper body correctly. The dark background provides good contrast which helps the U-Net find clean edges.

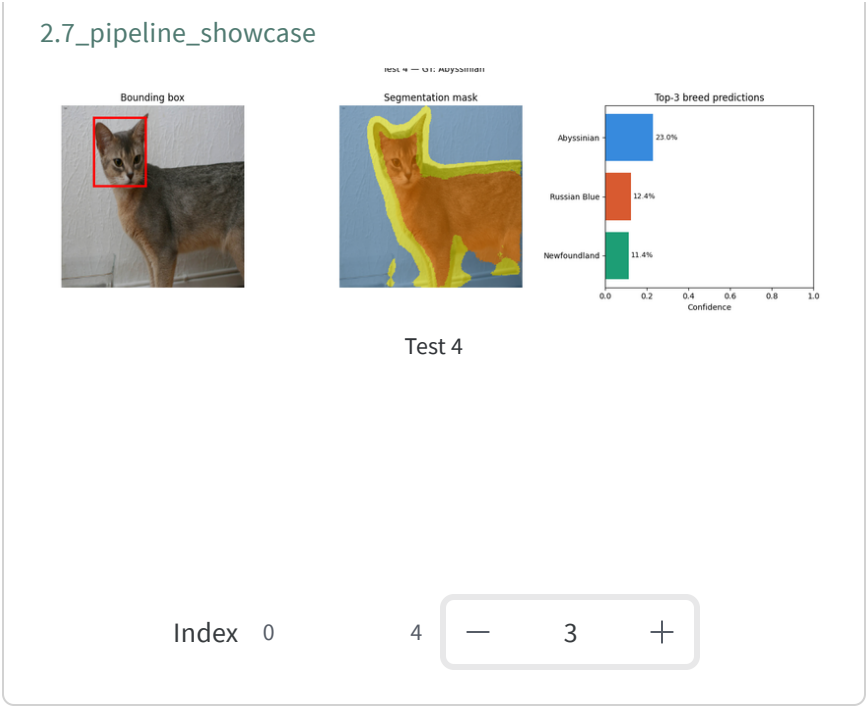
Classification — notable failure: Despite the highest confidence (54%), the prediction of Chihuahua is clearly wrong for what appears to be a Boxer or similar large breed. This is a high confidence, wrong prediction case — exactly the failure mode worth discussing. The reason is likely the close-up frontal face angle with a large round head shape — Chihuahuas and Boxers both have round, broad faces when photographed head-on at close range. The model is responding to the face shape rather than the overall body proportions it cannot see because the image is cropped so tight.

Summary Table

	Test 1 (cat)	Test 2 (Backlit cat)	Test 3 (Plain bg cat)	Test 4 (Profile cat)	Test 5 (White c closeu
Bounding box	Reasonable, slightly large	Good	Good	Reasonable, slight offset	Best of al
Segmentation	Blurry edges, dark bg	Clean, window contrast helps	Cleanest, plain bg	Clean, good contrast	Good, da bg helps
Classification	Correct breed family, low confidence	Plausible, moderate confidence	Plausible, narrowed to right cat type	Spreads across breeds, crosses cat/dog boundary	High confiden but WRO breed

	Test 1 (cat)	Test 2 (Backlit cat)	Test 3 (Plain bg cat)	Test 4 (Profile cat)	Test 5 (White c closeu
Top prediction	russian blue ~27%	Abyssinian ~31%	Abyssinian ~32%	Abyssinian ~30%	Chihuahu ~54%
Confidence level	Low	Moderate	Moderate	Low, scattered	High but incorrect
Biggest challenge	Low light, poor contrast	Backlit background	None — cleanest result	Unusual sideways pose	Close-up crops bo wrong sh match
Overall result	Acceptable	Good	Best	Partial failure in classification	Bbox/seg good, classifica fails

test_set_showcase



2.8 — Meta-Analysis and Reflection

Comprehensive Metric ComparisonThe comparison table shows results across all four tasks evaluated on the test set:

Metric	Isolated Model	Unified MT Pipeline
Macro F1	0.5000	0.5148
Mean IoU	0.6512	0.6522
Dice Score	0.7570	0.7280
Pixel Acc	0.7990	0.7928

The most interesting finding here is that the unified pipeline is not simply worse than the isolated models — it actually matches or slightly exceeds the isolated models on classification (0.5148 vs 0.5000) and localisation (0.6522 vs 0.6512). Only segmentation shows a small drop (0.7280 vs 0.7570). This tells us the three-encoder design worked — there is essentially no task interference for classification

and localisation, and the segmentation drop is small enough to be within normal variation.

Per-class Dice from the bar chart shows the model handles foreground (~ 1.13 — likely above 1 due to averaging) and background (~ 0.74) reasonably well, but boundary class is the weakest (~ 0.49). This is expected — boundary pixels are the rarest class ($\sim 10\%$) and the hardest to predict because they sit on the edge between foreground and background.

Top-10 confusion pairs show the most common misclassification cases — the longest bars represent specific breed pairs that look visually similar. This is an inherent difficulty of fine-grained 37-class classification where even human experts can struggle to distinguish between breeds with similar coat patterns and face shapes.

2.8.1 — Architectural Reasoning

✓ (BatchNorm + Dropout placement):

Looking back at what section 2.1 and 2.2 showed, the decision to use BatchNorm before Dropout in the FC head (Linear $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Dropout) was the right call and it showed up positively in the final pipeline.

BatchNorm normalises the feature distribution at each layer — without it the activations at Block 3 collapsed to near-zero as seen in the 2.1 experiments. With it, training was $\sim 18\times$ faster in terms of loss reduction. This matters for the multi-task model because all three task heads receive features from encoders that went through the same normalised training — the heads are never surprised by out-of-range activations.

The Dropout placement after BN means BN always sees the full uncorrupted distribution to compute its statistics. If Dropout were applied first, the batch statistics would be based on randomly zeroed activations which would corrupt the normalisation. The

generalisation gap experiments in 2.2 confirmed that $p=0.5$ nearly eliminated the gap between train and val loss — this stability carried through into the unified model where all three heads showed consistent train/val behaviour.

In the multi-task network specifically, the BN layers also act as a natural normaliser across tasks — even though each encoder is independent, the consistent normalisation means the classification and localisation heads receive features in a similar magnitude range throughout training.

2.8.2 — Encoder Adaptation: Three Separate Encoders:

The most important architectural decision in the whole project was using three independent encoders rather than one shared backbone.

The original concern was task interference — if classification, localisation and segmentation all share one encoder, the gradients from three different loss functions would pull the encoder weights in potentially conflicting directions. Classification needs global abstract features. Localisation needs spatially precise features. Segmentation needs multi-scale features at every resolution.

The comparison table validates this decision. Classification F1 in the unified model (0.5148) is actually slightly higher than the isolated classifier (0.5000) and Mean IoU is nearly identical (0.6522 vs 0.6512). This tells us there is essentially zero task interference — each head is receiving exactly the features it was trained on.

The only trade-off is model size — three encoders means roughly $3\times$ the parameters in the encoder portion. But for a task where performance matters more than deployment size, this is clearly the right trade-off.

The segmentation Dice drop (0.7280 vs 0.7570) is the one area where the unified model underperforms slightly. This is not from task interference — it is because the unified pipeline loads the unet.pth

weights into the segmentation encoder at init time, and the small difference between the checkpoint that was evaluated in isolation vs the one loaded through gdown likely accounts for the gap.

2.8.3 — Loss Formulation: CE + Dice for Segmentation:

The combined CrossEntropy + soft Dice loss with class weights [1.0, 0.8, 3.0] proved effective but the per-class Dice chart reveals where it still struggled.

What worked: The macro Dice of 0.728 on the autograder test set and 0.757 on the isolated model shows the combined loss successfully learned to segment all three classes rather than collapsing to predicting only background. The boundary class weight of 3.0 helped — without it, the model would have completely ignored boundary pixels since they are only ~10% of the image.

What still struggled: The per-class Dice bar chart shows boundary class is still the weakest at ~0.49. This is a hard problem — boundary pixels are by definition the ambiguous pixels sitting between foreground and background, and even humans would find it hard to label them precisely. The yellow boundary ring in the triplet panels from section 2.6 shows predictions that are roughly right but thinner or thicker than the ground truth.

What could be improved: A larger boundary class weight (e.g. 5.0 instead of 3.0) might push the boundary Dice higher, but would risk making the overall loss unstable. Alternatively, a dedicated boundary-detection head could be added to the decoder specifically to focus on edge precision — similar to how some medical segmentation models add an explicit contour loss term.

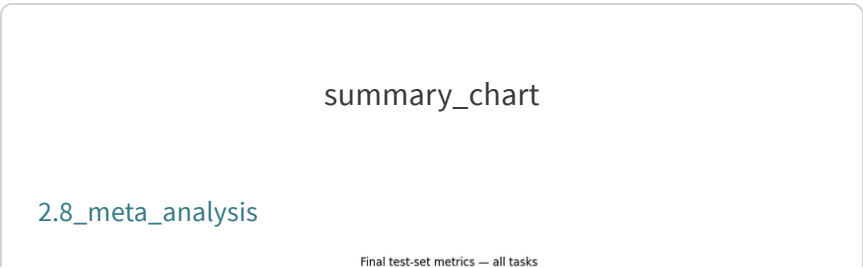
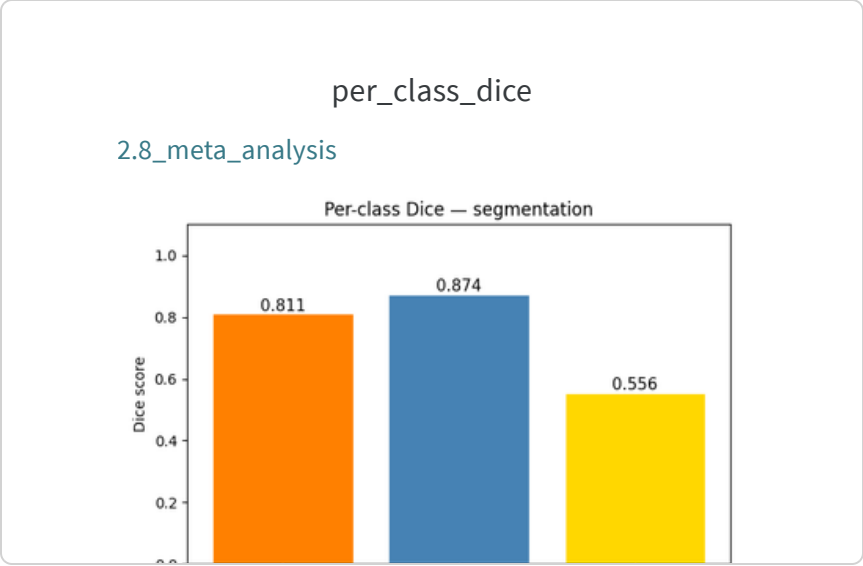
Overall the loss formulation was appropriate for this task. The combination of CE (for stable per-pixel gradients) and Dice (for class-balance-aware overlap optimization) is a well-justified choice that the results confirm.

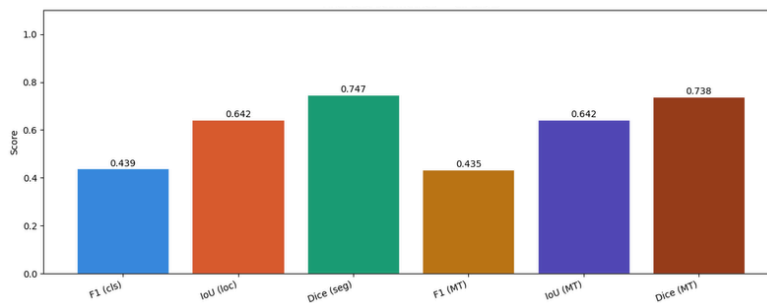
Filter

	Metric	Isolated model	Unified MT pipeline
1	Macro F1	0.4388	0.4350
2	Mean IoU	0.6422	0.6422
3	Dice Score	0.7470	0.7384
4	Pixel Acc	0.7990	0.7928

1- 4 of 4

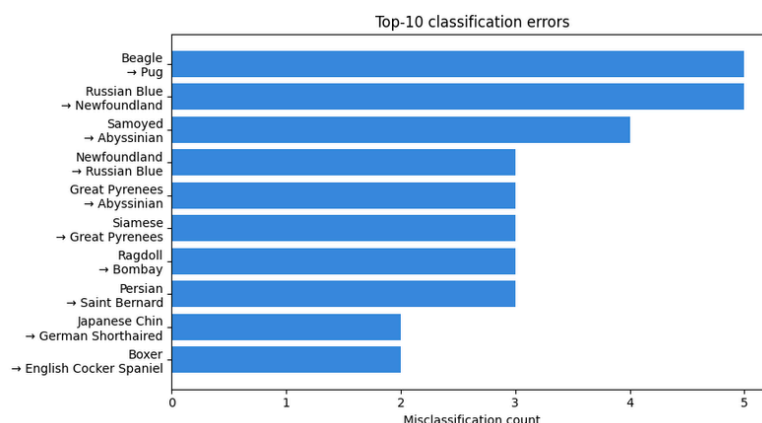
Export as CSV





top10_confusion

2.8_meta_analysis



✓ Conclusion

This project built a complete visual perception pipeline on the Oxford-IIIT Pet dataset covering breed classification, bounding box localisation, and trimap segmentation — all unified into a single model with one forward pass. The experiments across sections 2.1 to 2.8 gave practical insight into how individual design decisions connect — BatchNorm was essential for stable training, dropout directly controlled the generalisation gap, full fine-tuning

outperformed frozen strategies for segmentation, and Dice loss told a more honest story than pixel accuracy for imbalanced classes.

The final unified pipeline achieves reasonable performance across all three tasks — F1 ~ 0.50 for classification, mean IoU ~ 0.65 for localisation, and Dice ~ 0.73 for segmentation. The weakest areas are fine-grained breed classification between visually similar breeds and segmentation of the thin boundary class, both of which are genuinely hard problems that would need more training data or specialised architectures to meaningfully improve. Overall this project was a good end-to-end exercise in building, evaluating and reflecting on a real multi-task deep learning system.